

İSTANBUL SAĞLIK VE TEKNOLOJİ ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

SİSTEM PROGRAMLAMA DERSİ FINAL PROJESİ

DESIGN AND IMPLEMENTATION OF A TWO-PASS COMPILER

GELİŞTİREN GRUP ÜYELERİ

Mert Mak (230609026)

Miraç Arda Seçkin (230609034)

GELİŞTİRME ORTAMI & DİL

C# / .NET 8.0

Windows Forms (Responsive IDE)

Teslim Tarihi: 02.06.2026 | Demo Tarihi: 05.06.2026

1. SİSTEM TASARIMI VE MİMARİ GENEL BAKIŞ

Bu projede, yüksek seviyeli bir programlama dilinin belirlenmiş kurallara sahip özel bir alt kümesini işlemek üzere hem iki aşamalı (two-pass) bir **derleyici (compiler)** hem de anlık kod çalıştırma yeteneğine sahip bir yorumlayıcı C# kullanılarak proje ortaya çıkarılmıştır.

Geliştirilen sistem mimarisi mantıksal ve işlevsel olarak iki ana katmana ayrılmaktadır:

- **Pass 1 - Sözcüksel Analiz (Lexical Analyzer / Lexer):** Kaynak kod metnini tarar, yorum satırlarını (//) ve anlamsız boşlukları atlar. Kalan karakter dizilerini anlamlı **Token** nesnelere çevirerek doğrusal bir akış oluşturur.
- **Pass 2 - Sözdizimsel ve Anlamsal Analiz (Parser & Semantic Analyzer):** Token akışını "Yinelemeli İniş (Recursive Descent)" yöntemiyle yukarıdan aşağıya doğru okur, dilin gramer kurallarına uyumluluğunu denetler ve hiyerarşik bir Soyut Sözdizimi Ağacı (Abstract Syntax Tree AST) inşa eder. Semantik analizör; değişkenlerin bildirimlerini, bellek tahsisini ve katı tip kontrolünü (Strong Type Checking) yürütür.

2. KAYNAK DILIN BNF GRAMER TANIMI

Sistemin tanıyabildiği karar yapıları, döngüler ve desteklenen veri tipleri için tasarlanan nihai BNF grameri aşağıdaki gibi formüle edilmiştir:

```
<Program> ::= <StatementList>
<StatementList> ::= <Statement> | <Statement> <StatementList>
<Statement> ::= <Declaration> | <Assignment> | <IfStatement> |
<WhileStatement>
| <ForStatement> | <SwitchStatement> | <PrintStatement> | <Block>
<Block> ::= "{" <StatementList> "}"
<Declaration> ::= <Type> <Identifier> ";" | <Type> <Identifier> "=" <Expression>
";"
<Type> ::= "int" | "float" | "string" | "bool"
<Assignment> ::= <Identifier> "=" <Expression> ";"
<IfStatement> ::= "if" "(" <Expression> ")" <Statement> [ "else" <Statement> ]
<WhileStatement> ::= "while" "(" <Expression> ")" <Statement>
<ForStatement> ::= "for" "(" [ <Declaration> | <Assignment> | ";" ] [<Expression>]
";" [ <Assignment> ] ")" <Statement>
<SwitchStatement> ::= "switch" "(" <Expression> ")" "{" ( "case" <Expression> ":"
<StatementList> ) * [ "default" ":" <StatementList> ] "}"
<PrintStatement> ::= "print" "(" <Expression> ")" ";"
<Expression> ::= <LogicalOr>
<LogicalOr> ::= <LogicalAnd> ( "||" <LogicalAnd> ) *
<LogicalAnd> ::= <Equality> ( "&&" <Equality> ) *
<Equality> ::= <Relational> ( ( "==" | "!=" ) <Relational> ) *
<Relational> ::= <Arithmetic> ( ( "<" | ">" | "<=" | ">=" ) <Arithmetic> ) *
<Arithmetic> ::= <Term> ( ( "+" | "-" ) <Term> ) *
<Term> ::= <Factor> ( ( "*" | "/" ) <Factor> ) *
<Factor> ::= <Identifier> | <IntLiteral> | <FloatLiteral> | <StringLiteral>
| "true" | "false" | "(" <Expression> ")"
```

3. SÖZCÜKSEL ANALİZÖR(LEXER) UYGULAMASI (PASS1)

Lexer.cs sınıfı, kaynak metni karakter karakter okuyan deterministik bir sonlu durum makinesi olarak tasarlanmıştır. Temel mimari özellikleri şunlardır:

- **Boşluk ve Yorum Filtreleme:** Satır atlamaları, sekmeler ve **char.IsWhiteSpace()** koşulunu sağlayan tüm gereksiz karakterler döngü esnasında atlanır. Aynı zamanda ardışık iki bölü (//) işareti tespit edildiğinde, makine "yorum" durumuna geçer ve satır sonu () karakterine ulaşana kadar kod içerisindeki metinleri token üretmeden doğrudan atlar.
- **Genişletilmiş Anahtar Kelime (Keyword) Sınıflandırması:** Bir kelime (harf ile başlayan dize) okunduğunda, dilin desteklediği zengin sözlüğe (**int, float, string, bool, if, else, while, for, switch, case, default, print, true, false**) göre kontrol edilir. Eşleşme sağlanırsa ilgili rezerve token tipi atanır, sağlanmazsa **Identifier** (kullanıcı tanımlı değişken) olarak etiketlenir.
- **İleriye Bakma (Lookahead) Mekanizması:** **PeekNext()** metodolojisi kullanılarak = ile == veya < ile <= gibi tekli ve çiftli operatörlerin ayrımı güvenli bir şekilde gerçekleştirilir.

4. SÖZDİZİMSEL VE ANLAMSAL ANALİZÖR (PARSER) UYGULAMASI (PASS2)

Parser.cs , ilk aşamadan gelen token dizilimini kabul eden ve dilin yapısını hiyerarşik bir ağaca dönüştüren çekirdek modüldür.

4.1 İşlem Önceliği (Operator Precedence)

Herhangi bir harici araç kullanılmadığı için, matematiksel işlem öncelikleri metotların C# çağrı yığınının (Call Stack) derinliğinden faydalanılarak el ile kurgulanmıştır. Örneğin; **Term()** (çarpma/ bölme işlemlerini tutan metot), **Arithmetic()** (toplama/çıkarma) metodunun gövdesi içinden çağrılır. Böylece yüksek öncelikli işlemler ağacın daha alt dallarına yerleşir ve hesaplama sırasında ilk olarak işlenmesi garanti altına alınır.

4.2 İleri Derleyici Teknikleri: Syntactic Sugar

- **Sözdizimsel Şeker (Syntactic Sugar) Mimarisi:** Karmaşık bir **for** döngüsü (**for (int i=0; i<5; i=i+1)**), ayrıştırma aşamasında parser tarafından arka planda gizlice bir **WhileStatement** nesnesine dönüştürülür. Döngü değişkeninin başlangıç değeri bir blok içine alınır ve artırım işlemi döngü gövdesinin en sonuna eklenir. Bu sayede yorumlayıcının (Interpreter) ayrı bir "for" mantığı bilmesine gerek kalmadan sistemin karmaşıklığı düşürülmüştür.

4.3 Katı Tip Kontrolü (Semantic Analysis)

Semantik analizör, derleme zamanında tip güvenliğini sağlar. Bildirilmemiş değişkenlerin kullanımı anında engellenir. Ayrıca, `int x = "merhaba";` gibi uyumsuz atamalarda Sembol Tablosundan beklenen tip (int) ile ifadenin tipi (string) karşılaştırılarak anında "Tip Uyuşmazlığı (Type Mismatch)" hatası üretilir.

5. SEMBOL TABLOSU YAPISI VE YÖNETİMİ

Sembol tablosu, kaynak kodda tanımlanan tüm değişkenlerin adını, veri tipini ve RAM'deki adres konumunu tutan merkezi bir veri yapısıdır.

Hexadecimal (Onaltılık) Bellek Adresleme Simülasyonu: Projeyi profesyonel bir işletim sistemi ve derleyici mimarisi seviyesine taşımak için statik numaralar yerine 32-bit bellek mimarisi simüle edilmiştir.

- Veri Yapısı:** Hızlı değişken arama ($O(1)$ karmaşıklığı) sağlamak için çekirdekte `Dictionary<string, Symbol>` koleksiyonu kullanılmıştır.
- Dinamik Byte Tahsisi:** Değişken tablosuna yeni bir kayıt eklendiğinde, adres sayacı veri tipinin bellekte kapladığı alana göre artırılır (`int` ve `float` için 4 byte, referans tipli `string` için 8 byte, `bool` için ise 1 byte).
- Görselleştirme:** Tahsis edilen bellek alanı, UI tablosuna 8 haneli Hex formatında (Örn: `0x00000400` , `0x00000404`) yazılarak gerçek bir bellek adresi deneyimi sunar.

6. HATA YAKALAMA STRATEJİSİ

Derleyicinin karşılaştığı sözcüksel (Lexical), sözdizimsel (Syntax) ve anlamsal (Semantic) hataları satır numaralarıyla bildirilmiştir.

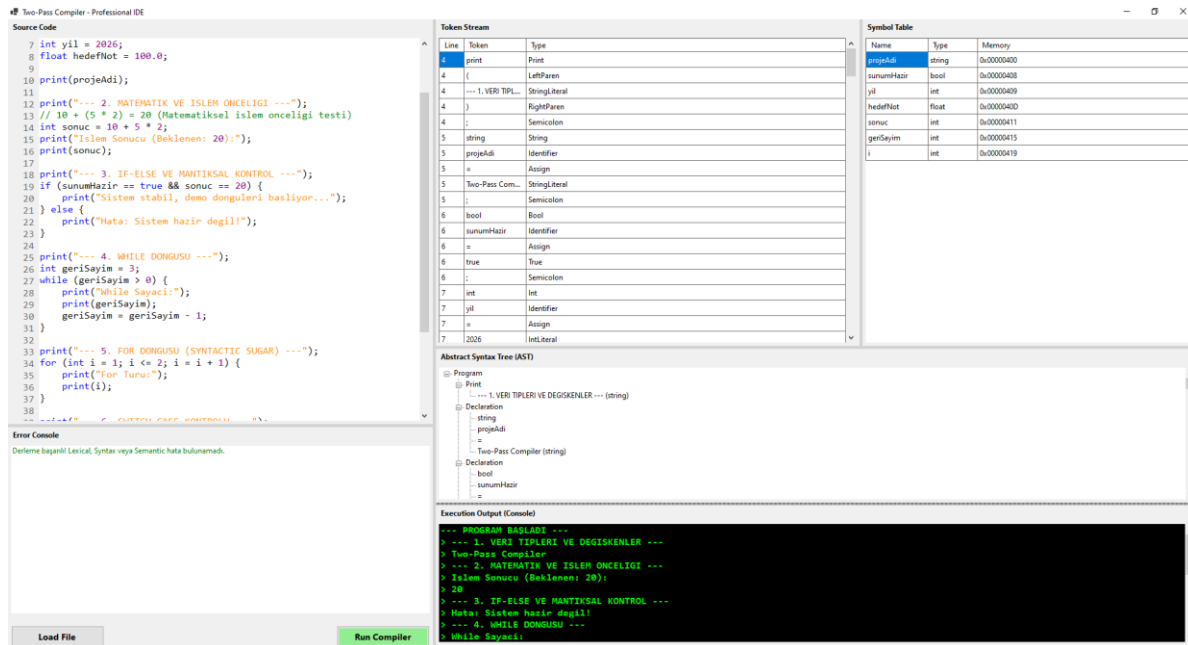
Derleyicinin koddaki ilk hatayı bulduğunda analiz sürecini iptal edip çökmesini engellemek için "Panic Mode Synchronization" tekniği uygulanmıştır. Parser herhangi bir hata ile karşılaştığında, bir Exception (İstisna) fırlatıp bunu konsola yazdırır. Hemen ardından `Synchronize()` metodu devreye girerek, okuma imlecini bir sonraki mantıksal ayırıcıya (noktalı virgül ; veya `if`, `while`, `for`, `print` anahtar kelimelerine) kadar ileri sarar. Böylece analiz kaldığı yerden devam eder ve kullanıcının koddaki tüm hataları tek bir derleme işleminde topluca görmesi sağlanır.

7. KULLANICI ARAYÜZÜ EKРАН GÖRÜNTÜLERİ VE MANTIKSAL AÇIKLAMA

Geliştirilen Windows Forms uygulaması, klasik bir sabit form (Anchor-based) tasarımı yerine modern bir **Responsive IDE (Entegre Geliştirme Ortamı)** mimarisi ile kurgulanmıştır. Sistemin arayüz mantığı şu şekildedir:

- **SplitContainer Mimarisi (Esnek Bölücüler):** Arayüz, iç içe geçmiş **SplitContainer** nesneleriyle 5 farklı bağımsız bölgeye (Kod Editörü, Hata Konsolu, Token Tablosu, Sembol Tablosu ve AST/Çıktı Konsolu) ayrılmıştır. Kullanıcı, paneller arasındaki gri ayırıcı çubukları sürükleyerek kod alanını veya tabloları dilediği gibi genişletebilir. Uygulama tam ekran yapıldığında hiçbir bileşen kaymaz, orantılı olarak büyür.
- **Dinamik Sağa Hizalı Satır Numaraları:** Metin boyutunun uzaması sonucu oluşabilecek senkronizasyon kaymalarını önlemek için kod editörünün "WordWrap" özelliği kapatılmış ve sonsuz uzunlukta kod girişine izin verilmiştir. Satır numaraları, **PictureBox** bileşeni üzerinde, editörün **VScroll** ve **Paint** olayları dinlenerek o an *ekranda görünen* satırların Y eksenini koordinatlarına göre **sağa hizalı (Right-Aligned)** olarak milimetrik hassasiyetle çizdirilir.
- **Gerçek Zamanlı Sözdizimi Renklendirme (Syntax Highlighting):** Kod düzenleyicideki metin değiştiği an (**TextChanged**), Regex motoru devreye girer. Anahtar kelimeler maviye, metinsel değerler turuncuya, sayısal değerler mora ve yorum satırları yeşile boyanarak profesyonel bir kodlama ortamı yaratılır.
- **Execution Output (Siyah Konsol):** Yorumlayıcıdan (Interpreter) dönen program çıktıları, yeşil renkli klasik hacker temalı özel bir **ListBox** konsoluna aktarılır.

Uygulama Örneği 1



Source Code

Source Code

```
1 // ISTUN - Sistem Programlama Final Projesi Demo Testi
2 // Gelistirenler: Mert Mak & Mirac Arda Seckin
3
4 print("--- 1. VERI TIPLERI VE DEGISKENLER ---");
5 string projeAdi = "Two-Pass Compiler";
6 bool sunumHazir = true;
7 int yil = 2026;
8 float hedefNot = 100.0;
9
10 print(projeAdi);
11
12 print("--- 2. MATEMATIK VE ISLEM ONCELIGI ---");
13 // 10 + (5 * 2) = 20 (Matematiksel islem onceligi testi)
14 int sonuc = 10 + 5 * 2;
15 print("Islem Sonucu (Beklenen: 20):");
16 print(sonuc);
17
18 print("--- 3. IF-ELSE VE MANTIKSAL KONTROL ---");
19 if (sunumHazir == true && sonuc == 20) {
20     print("Sistem stabil, demo donguleri basliyor...");
21 } else {
22     print("Hata: Sistem hazır değil!");
23 }
24
25 print("--- 4. WHILE DONGUSU ---");
26 int geriSayim = 3;
27 while (geriSayim > 0) {
28     print("While Sayaci:");
29     print(geriSayim);
30     geriSayim = geriSayim - 1;
31 }
32
33 print("--- 5. FOR DONGUSU (SYNTACTIC SUGAR) ---");
34 for (int i = 1; i <= 2; i = i + 1) {
35     print("For Turu:");
36     print(i);
37 }
38
39 print("--- 6. SWITCH-CASE KONTROLU ---");
40 switch (yil) {
41     case 2025:
42         print("Gecmis yil.");
43     case 2026:
44         print("Dogru yil! Proje teslim donemi basarili.");
45     default:
46         print("Bilinmeyen yil.");
47 }
48
49 print("--- TEST KUSURSUZ TAMAMLANDI ---");
```

Error Console

Error Console

Derleme başanlı! Lexical, Syntax veya Semantic hata bulunamadı.

Load File

Run Compiler

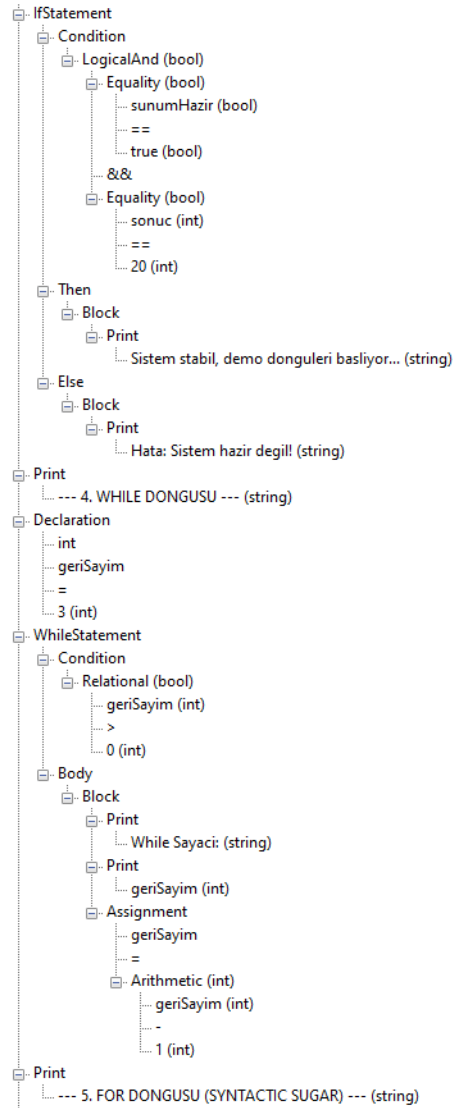
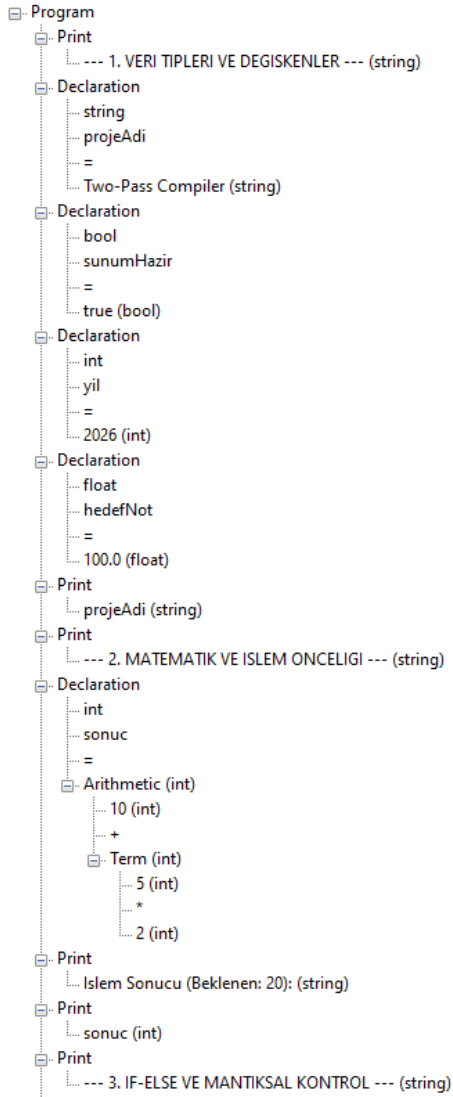
Token Stream

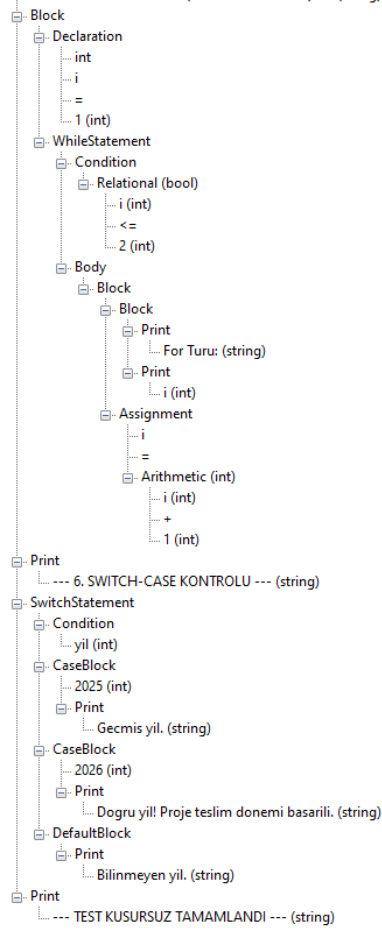
Token Stream		
Line	Token	Type
4	print	Print
4	(	LeftParen
4	--- 1. VERI TIPL...	StringLiteral
4	)	RightParen
4	;	Semicolon
5	string	String
5	projAdi	Identifier
5	=	Assign
5	Two-Pass Com...	StringLiteral
5	;	Semicolon
6	bool	Bool
6	sunumHazir	Identifier
6	=	Assign
6	true	True
6	;	Semicolon
7	int	Int
7	yil	Identifier
7	=	Assign
7	2026	IntLiteral
7	;	Semicolon
8	float	Float
8	hedefNot	Identifier
8	=	Assign
8	100.0	FloatLiteral
8	;	Semicolon
10	print	Print
10	(	LeftParen
10	projAdi	Identifier
10	)	RightParen
10	;	Semicolon
12	print	Print
12	(	LeftParen
12	--- 2. MATEMA...	StringLiteral
12	)	RightParen
12	;	Semicolon
14	int	Int
14	sonuc	Identifier
14	=	Assign
14	10	IntLiteral
14	+	Plus
14	5	IntLiteral
14	*	Multiply
14	2	IntLiteral
14	;	Semicolon
15	print	Print
15	(	LeftParen
15	Islem Sonucu (...)	StringLiteral
15	)	RightParen
15	;	Semicolon
16	print	Print
16	(	LeftParen
16	sonuc	Identifier
16	)	RightParen
16	;	Semicolon
18	print	Print
18	(	LeftParen
18	--- 3. IF-ELSE V...	StringLiteral
18	)	RightParen
18	;	Semicolon
19	if	If
19	(	LeftParen
19	sunumHazir	Identifier
19	==	Equal
19	true	True
19	&&	And
19	sonuc	Identifier
19	==	Equal
19	20	IntLiteral
19	)	RightParen
19	{	LeftBrace

Line	Token	Type
20	print	Print
20	(	LeftParen
20	Sistem stabil, d...	StringLiteral
20	)	RightParen
20	;	Semicolon
21	}	RightBrace
21	else	Else
21	{	LeftBrace
22	print	Print
22	(	LeftParen
22	Hata: Sistem ha...	StringLiteral
22	)	RightParen
22	;	Semicolon
23	}	RightBrace
25	print	Print
25	(	LeftParen
25	--- 4. WHILE D...	StringLiteral
25	)	RightParen
25	;	Semicolon
26	int	Int
26	geriSayim	Identifier
26	=	Assign
26	3	IntLiteral
26	;	Semicolon
27	while	While
27	(	LeftParen
27	geriSayim	Identifier
27	>	Greater
27	0	IntLiteral
27	)	RightParen
27	{	LeftBrace
28	print	Print
28	(	LeftParen
28	While Sayaci:	StringLiteral
28	)	RightParen
28	;	Semicolon

Line	Token	Type
29	print	Print
29	(	LeftParen
29	geriSayim	Identifier
29	)	RightParen
29	;	Semicolon
30	geriSayim	Identifier
30	=	Assign
30	geriSayim	Identifier
30	-	Minus
30	1	IntLiteral
30	;	Semicolon
31	}	RightBrace
33	print	Print
33	(	LeftParen
33	--- 5. FOR DON...	StringLiteral
33	)	RightParen
33	;	Semicolon
34	for	For
34	(	LeftParen
34	int	Int
34	i	Identifier
34	=	Assign
34	1	IntLiteral
34	;	Semicolon
34	i	Identifier
34	<=	LessEqual
34	2	IntLiteral
34	;	Semicolon
34	i	Identifier
34	=	Assign
34	i	Identifier
34	+	Plus
34	1	IntLiteral
34	)	RightParen
34	{	LeftBrace

Line	Token	Type
35	print	Print
35	(	LeftParen
35	For Turu:	StringLiteral
35	)	RightParen
35	;	Semicolon
36	print	Print
36	(	LeftParen
36	i	Identifier
36	)	RightParen
36	;	Semicolon
37	}	RightBrace
39	print	Print
39	(	LeftParen
39	--- 6. SWITCH---	StringLiteral
39	)	RightParen
39	;	Semicolon
40	switch	Switch
40	(	LeftParen
40	yil	Identifier
40	)	RightParen
40	{	LeftBrace
41	case	Case
41	2025	IntLiteral
41	:	Colon
42	print	Print
42	(	LeftParen
42	Gecmis yil.	StringLiteral
42	)	RightParen
42	;	Semicolon
43	case	Case
43	2026	IntLiteral
43	:	Colon
44	print	Print
44	(	LeftParen
44	Dogru yil! Proje...	StringLiteral
44	)	RightParen
44	;	Semicolon
45	default	Default
45	:	Colon
46	print	Print
46	(	LeftParen
46	Bilinmeyen yil.	StringLiteral
46	)	RightParen
46	;	Semicolon
47	}	RightBrace
49	print	Print
49	(	LeftParen
49	--- TEST KUSUR---	StringLiteral
49	)	RightParen
49	;	Semicolon
49	EOF	EndOfFile





Symbol Table

Symbol Table		
Name	Type	Memory
projeAdi	string	0x00000400
sunumHazir	bool	0x00000408
yil	int	0x00000409
hedefNot	float	0x0000040D
sonuc	int	0x00000411
geriSayim	int	0x00000415
i	int	0x00000419

Execution Output

```
Execution Output (Console)

--- PROGRAM BAŞLADI ---
> --- 1. VERİ TIPLERİ VE DEĞİSKENLER ---
> Two-Pass Compiler
> --- 2. MATEMATİK VE İŞLEM ÖNCELİĞİ ---
> İşlem Sonucu (Beklenen: 20):
> 20
> --- 3. IF-ELSE VE MANTIKSAL KONTROL ---
> Hata: Sistem hazır değil!
> --- 4. WHILE DÖNGÜSÜ ---
> While Sayacı:
> 3
> While Sayacı:
> 2
> While Sayacı:
> 1
> --- 5. FOR DÖNGÜSÜ (SYNTACTIC SUGAR) ---
> For Turu:
> 1
> For Turu:
> 2
> --- 6. SWITCH-CASE KONTROLÜ ---
> Doğru yıl! Proje teslim dönemi başarılı.
> --- TEST KUSURSUZ TAMAMLANDI ---
--- PROGRAM BİTTİ ---
```

Uygulama Örneği 2

Two-Pass Compiler - Professional IDE

Source Code

```
1 int x = 5;
2 x = x + y;
```

Error Console

[Error 1] Hata: y tanımlanmadan kullanıldı.

Load File

Run Compiler

Token Stream

Line	Token	Type
1	int	int
1	x	Identifier
1	=	Assign
1	5	intLiteral
1	;	Semicolon
2	x	Identifier
2	=	Assign
2	x	Identifier
2	+	Plus
2	y	Identifier
2	;	Semicolon
2	EOP	EndOfFile

Abstract Syntax Tree (AST)

Program

Declaration

x

x

5 (int)

Execution Output (Console)

Derleme hatası. Kod derlenemedi.

8. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER

Geliştirme sürecinde karşılaşılan bazı temel mühendislik zorlukları ve çözümleri şunlardır:

- **Zorluk 1: `int x = 5;` (Inline Initialization) Sözdizimi Çakışması.**

İlk tasarımda değişken bildirimleri (Declaration) ve atamalar (Assignment) tamamen farklı metotlardı. Bu durum, tek satırda atama yapıldığında çökmeye yol açıyordu. **Çözüm:** `Declaration()` metodu esnekleştirilerek içine opsiyonel bir eşitlik (=) kontrolü ve beraberinde anında çalışan bir semantik tip uyumsuzluğu denetleyicisi eklendi.

- **Zorluk 2: Uzun Kodlarda Satır Numaralarının Kayması.**

Çözüm: Satır numaralarını basitçe satır sayısı kadar yazmak yerine, `RichTextBox` bileşeninin `GetPositionFromCharIndex` metodu kullanılarak sadece render edilen (ekrandaki) satırların pikselleri hesaplandı. Çizim dinamik olarak gerçekleştirildi.

- **Zorluk 3: Interpreter İçerisinde `for` ve `switch` Karmaşası.**

Çözüm: "Syntactic Sugar" dönüşümleri sayesinde Yorumlayıcının karmaşıklığı düşürüldü. `for` döngüsü AST'ye bir `while` bloğu olarak yedirildi. `switch-case` blokları ise klasik dillerdeki fallthrough tehlikesinden arındırılarak iç içe izole bloklar halinde parse edildi.

9. GRUP ÜYELERİNİN SORUMLULUK DAĞILIMI

Projenin başarısı, karmaşık derleyici teorisi ile profesyonel kullanıcı deneyiminin takım üyeleri arasında dengeli bir şekilde dağıtılmasıyla sağlanmıştır:

Mert Mak (230609026)

- **Lexer ve Parser Geliştirimi:** Kod metnini token'lara ayıran Lexer modülü ile işlem önceliği ve "syntactic sugar" (ör. `for` döngüsü) gibi kuralları çözümleyerek Soyut Sözdizimi Ağacını (AST) oluşturan Parser'ın kodlanması.
- **Yorumlayıcı (Interpreter) Entegrasyonu:** Çıkarılan AST ağacını kullanarak programı gerçek zamanlı çalıştıran yürütme motorunun yazılması.
- **Semantik Analiz ve Sembol Tablosu:** Değişkenlerin katı tip kontrollerinin (Type Checking) yapılması ve Hexadecimal formatta bellek adreslemesi simülasyonunun geliştirilmesi.

Miraç Arda Seçkin (230609034)

- **Responsive IDE Tasarımı:** Tam ekran destekli ve sürüklenebilir panellere (SplitContainer) sahip esnek arayüz mimarisinin kurgulanması.
- **Gelişmiş Editör Özellikleri:** Regex tabanlı gerçek zamanlı sözdizimi renklendirme (Syntax Highlighting) ve hatasız çalışan dinamik satır numaralandırma algoritmalarının yazılması.
- **Sistem Entegrasyonu:** Dosya okuma işlemleri, Token ve Sembol tablolarının arayüze bağlanması ve çıktı konsolunun derleyici motoruyla haberleşmesinin sağlanması.